Informe Técnico: Gestión de Procesos y Planificación de CPU

Análisis e Implementación de un Mecanismo Interno de un Sistema Operativo

Curso: Sistemas Operativos
Programa: Ingeniería de Software
Modalidad: Proyecto Final de Curso
Mecanismo Seleccionado: Opción 1 - Gestión de procesos y planificación de CPU

---

1. Descripción del Problema

En los sistemas operativos modernos de propósito general y en entornos de servidores empresariales (como nodos de cómputo en la nube, servidores de aplicaciones web y microservicios), la CPU es un recurso compartido crítico. En estos entornos convergen procesos con naturalezas y requerimientos de servicio radicalmente dispares: 1. **Peticiones Interactivas y de Red (Web/API):** Tareas con ráfagas de CPU ultracortas que exigen tiempos de respuesta inmediatos para cumplir acuerdos de nivel de servicio (SLA). 2. **Procesos I/O-Bound:** Peticiones que alternan ciclos breves de cómputo con esperas prolongadas de acceso a almacenamiento secundario, bases de datos o interfaces de red. 3. **Procesos Batch CPU-Bound:** Procesamiento asíncrono de reportes, transformaciones analíticas, transcodificación o criptografía que demandan la CPU de forma continua y prolongada.

Conflicto Técnico Central

Cuando el planificador de la CPU no adopta una política adecuada: * **Efecto Convoy:** En algoritmos no apropiativos como FCFS, un proceso largo monopoliza la CPU mientras las tareas cortas se encolan detrás, degradando drásticamente el tiempo de espera promedio y la responsividad interactiva. * **Sobrecarga por Conmutación de Contexto (*Context Switch Overhead*):** En algoritmos apropiativos como Round Robin, un *quantum* excesivamente pequeño provoca una conmutación continua de registros y tablas del PCB, desperdiciando ciclos útiles de CPU (*thrashing*). Por el contrario, un *quantum* excesivamente grande degenera el algoritmo en FCFS. * **Inanición (*Starvation*):** En algoritmos basados en prioridad estricta o ráfaga restante (SRTF), un flujo continuo de procesos de alta prioridad o ráfaga corta condena a los procesos de baja prioridad a esperar indefinidamente, congelando tareas de fondo esenciales.

El problema abordado consiste en modelar, cuantificar experimentalmente y mitigar estos fenómenos mediante la implementación de un simulador determinista de gestión de procesos y algoritmos de planificación de CPU.

---

2. Objetivo del Proyecto

Diseñar, implementar y evaluar experimentalmente un simulador en C++17 que represente de forma fidedigna el ciclo de vida de los procesos y compare cuantitativamente las políticas de planificación de CPU (**FCFS, Round Robin, SRTF y Prioridades con Envejecimiento**).

Objetivos Específicos:

1. Implementar la estructura formal del **PCB (Process Control Block)** y modelar las transiciones completas entre los estados `NEW`, `READY`, `RUNNING`, `WAITING` y `TERMINATED`.
2. Modelar el subsistema de Entrada/Salida (E/S) para evaluar el bloqueo y desbloqueo concurrente de procesos.
3. Medir y comparar formalmente las métricas canónicas: Tiempo de Retorno (T), Tiempo de Espera (W), Tiempo de Respuesta (R), Utilización de CPU, Rendimiento (*Throughput*), costo de Cambio de Contexto y el Índice de Equidad de Jain.
4. Demostrar el fenómeno de **inanición** en escenarios de estrés y validar la efectividad de la técnica de **envejecimiento (*aging*)** como solución matemática y operativa.

---

3. Fundamentación del Mecanismo Seleccionado

3.1. El Bloque de Control de Proceso (PCB)

El PCB es la estructura fundamental de datos del kernel que mantiene el estado y metadatos de una tarea. En nuestra implementación, cada PCB almacena: * Identificador único de proceso (`PID`) y nombre descriptivo. * Vector de ráfagas secuenciales de CPU y E/S. * Estado del proceso (`ProcessState`). * Prioridad estática base y prioridad dinámica (modificada en tiempo de ejecución). * Temporizadores de llegada, primer despacho, terminación y acumuladores de espera. * Contador de cambios de contexto experimentados.

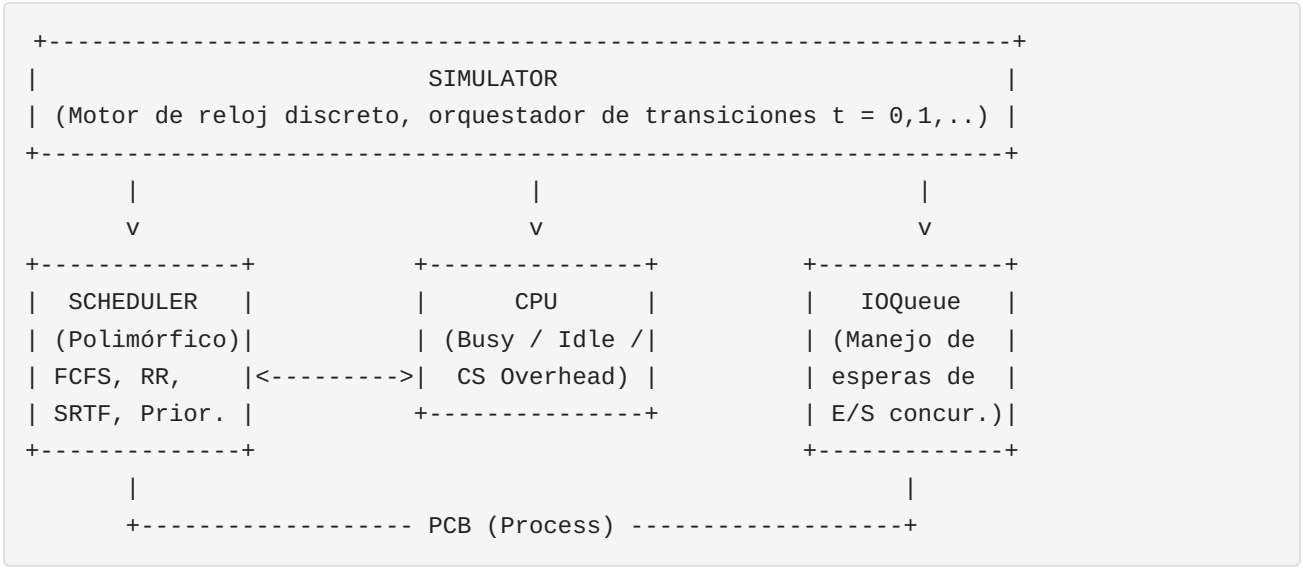
3.2. Ciclo de Vida y Transiciones de Estado

El motor de simulación modela una máquina de estados finitos determinista: * **NEW $\rightarrow$ READY:** El proceso alcanza su tiempo de llegada (T_{arribo}) y es admitido en la cola del planificador. * **READY $\rightarrow$ RUNNING:** El *Dispatcher* asigna la CPU al proceso seleccionado por la política activa. * **RUNNING $\rightarrow$ READY:** Ocurre por desalojo (*preemption*): expiración del *quantum* en Round Robin o arribo de un proceso con menor tiempo restante (SRTF) o mayor prioridad. * **RUNNING $ \rightarrow$ WAITING:** El proceso culmina su ráfaga actual de CPU y requiere una operación de E/S. * **WAITING $\rightarrow$ READY:** El subsistema de E/S finaliza la transferencia y reubica el proceso en la cola de listos. * **RUNNING $\rightarrow$ TERMINATED:** El proceso completa su última ráfaga, liberando recursos y registrando sus métricas finales.

---

4. Diseño de la Solución

La solución fue desarrollada en **C++17 modular**, sin dependencias externas de librerías propietarias, garantizando portabilidad, bajo acoplamiento y alta cohesión.



Componentes Principales:

- **`Process` (PCB):** Encapsula el estado, las ráfagas pendientes y el registro cronológico del proceso.
- **`Scheduler` (Interfaz Abstracta):** Define los métodos polimórficos `add_process()`, `get_next_process()`, `should_preempt()` y `on_tick()`.
- **`CPU`:** Simula el procesador físico. Aplica de manera explícita una penalización configurable de P ticks (`context_switch_penalty`) al conmutar entre diferentes procesos, modelando el costo de guardar y restaurar registros de hardware.
- **`IOQueue`:** Administra las solicitudes de E/S concurrentes, actualizando las ráfagas bloqueadas y notificando al simulador cuando una tarea debe regresar a la cola de listos.
- **`Metrics`:** Recolecta los tiempos ciclo a ciclo, comprime los intervalos para generar el Diagrama de Gantt en ASCII, calcula las medias estadísticas, el índice de equidad y exporta los datos a formato CSV.

---

5. Algoritmos y Estrategias Comparadas

1. **FCFS (*First-Come, First-Served*):**
2. *Naturaleza:* No apropiativo.
3. *Criterio de selección:* Cola FIFO estricta según orden de llegada.
4. *Comportamiento esperado:* Sencillez de implementación pero alta vulnerabilidad al efecto convoy.
5. **Round Robin (RR):**
6. *Naturaleza:* Apropiativo.

7. *Criterio de selección:* Turnos rotativos limitados por una porción temporal (*quantum* q).
8. *Comportamiento esperado:* Alta equidad y buena responsividad; balance sensible entre el valor de q y el costo del cambio de contexto.
9. **SRTF (*Shortest Remaining Time First*):**
10. *Naturaleza:* Apropiativo.
11. *Criterio de selección:* Menor tiempo restante en la ráfaga actual de CPU. Desaloja al proceso en ejecución si arriba uno con ráfaga menor.
12. *Comportamiento esperado:* Demostrablemente óptimo en tiempo de espera medio, pero susceptible a inanición de procesos largos.
13. **Prioridades con Envejecimiento (*Aging*):**
14. *Naturaleza:* Apropiativo.
15. *Criterio de selección:* Menor número entero de prioridad (1 = máxima prioridad).
16. *Mecanismo de Envejecimiento:* Si un proceso pasa K ciclos en estado `READY` sin ser atendido, su prioridad dinámica aumenta en 1 nivel. Esto garantiza una cota superior finita al tiempo de espera, erradicando la inanición.

---

6. Escenarios y Métricas de Prueba

6.1. Métricas de Evaluación

- **Tiempo de Retorno (*Turnaround Time*, T):** $T = T_{\text{terminación}} - T_{\text{llegada}}$
- **Tiempo de Espera (*Waiting Time*, W):** $W = T - \sum \text{Ráfagas CPU} - \sum \text{Ráfagas E/S}$
- **Tiempo de Respuesta (*Response Time*, R):** $R = T_{\text{primer despacho}} - T_{\text{llegada}}$
- **Utilización de CPU (%):** $(\text{Tiempo Útil de CPU} / \text{Tiempo Total}) \times 100$
- **Sobrecarga por Cambio de Contexto (%):** $(\text{Tiempo en Cambio de Contexto} / \text{Tiempo Total}) \times 100$
- **Rendimiento (*Throughput*):** $\text{Procesos Completados} / \text{Tiempo Total}$
- **Índice de Equidad de Jain (J):** $$J = \frac{\left(\sum_{i=1}^n x_i\right)^2}{n \sum_{i=1}^n x_i^2}, \quad \text{donde } x_i = \frac{\text{CPU}_i}{T_i}$$ Un valor $J = 1.0$ representa equidad matemática absoluta.

6.2. Escenarios de Prueba Implementados

- **Escenario 1 (Efecto Convoy):** Carga heterogénea donde un proceso batch de 35 ticks arriba en $t=0$, seguido de 4 peticiones web cortas (2 a 3 ticks) arribando en $t=1, 2, 3, 4$.
- **Escenario 2 (Procesos Mixtos y E/S):** Conjunto de 5 procesos que combinan múltiples ciclos `CPU -> E/S -> CPU`, demostrando el desacoplamiento de colas `READY` y `WAITING`.
- **Escenario 3 (Inanición y Aging):** Un proceso de fondo de baja prioridad compite contra una ráfaga sostenida de procesos urgentes de alta prioridad.

7. Resultados Obtenidos

Tabla 1: Resultados del Escenario 1 (Efecto Convoy)

Algoritmo	T_{prom} (ticks)	W_{prom} (ticks)	R_{prom} (ticks)	Utilización CPU	Overhead C.C.	Cambios Contexto	Jain Index
FCFS	39.60	30.00	30.80	91.67%	8.33%	4	0.29
Round Robin ($q=4$)	17.60	7.80	6.00	89.80%	10.20%	5	0.75
SRTF	14.80	5.00	3.20	89.80%	10.20%	5	0.88
Prioridad Estricta	16.40	6.40	4.60	88.00%	12.00%	6	0.74
Prioridad + Aging	22.80	12.80	11.60	88.00%	12.00%	6	0.66

Tabla 2: Resultados del Escenario 2 (Ciclo Completo con E/S)

Algoritmo	T_{prom} (ticks)	W_{prom} (ticks)	R_{prom} (ticks)	Utilización CPU	Overhead C.C.	Cambios Contexto	Jain Index
FCFS	18.20	5.40	2.40	71.43%	20.00%	7	0.76
Round Robin ($q=4$)	18.20	5.40	2.40	71.43%	20.00%	7	0.76
SRTF	17.80	5.00	2.20	71.43%	20.00%	7	0.77
Prioridad + Aging	18.40	5.60	2.20	71.43%	20.00%	7	0.77

Tabla 3: Resultados del Escenario 3 (Estrés de Inanición y Rescate por Aging)

Algoritmo	T_{prom} (ticks)	W_{prom} (ticks)	R_{prom} (ticks)	Proceso de Fondo: Fin	Proceso de Fondo: Espera
FCFS	30.00	22.71	23.57	Tick 25	0 ticks
Prioridad Estricta	13.29	5.86	3.00	Tick 52 (Último)	26 ticks (Inanición)
Prioridad con Aging	16.14	8.71	6.57	**Tick 47 (Rescatado)**	**21 ticks**

8. Análisis e Interpretación de Resultados

1. **Evidencia del Efecto Convoy:** En el Escenario 1, FCFS arrojó un tiempo de espera medio de 30.00 ticks y un índice de equidad paupérrimo (0.29). Las peticiones interactivas (`API_Login` , `API_Payment` , etc.) sufrieron esperas de más de 34 ticks esperando que `Analitica_Batch` terminara. SRTF redujo el tiempo de espera a solo 5.00 ticks (una mejora del 83.3%), elevando la equidad a 0.88.
2. **Impacto de la Conmutación de Contexto:** Cada desalojo genera un costo visible. En el Escenario 1, FCFS incurrió en 4 cambios de contexto (8.33% de sobrecarga), mientras que los algoritmos apropiativos alcanzaron entre 10.20% y 12.00% de tiempo consumido exclusivamente en conmutación de PCBs.
3. **Mecanismo Anti-Inanición:** En el Escenario 3, bajo Prioridad Estricta, la tarea de fondo fue desalojada inmediatamente en $t=1$ y forzada a esperar a que **todos** los procesos urgentes 1 al 6 terminaran (terminando en el tick 52). Con la activación del *Aging* ($K=5$), la prioridad dinámica del proceso de fondo se promovió gradualmente ($5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$), arrebatando la CPU a las tareas entrantes y culminando en el tick 47.

9. Análisis de Riesgos y Responsabilidades Profesionales

1. **Riesgo de Inanición en Sistemas en Tiempo Real y Críticos:** Un planificador sin mecanismos de envejecimiento puede provocar fallas catastróficas en sistemas embebidos o médicos si una rutina de telemetría o seguridad de baja prioridad nunca recibe tiempo de CPU.
2. **Denegación de Servicio (DoS) por Inyección de Tareas Cortas:** En sistemas con SRTF puro, un atacante que envíe ráfagas continuas de procesos mínimos puede paralizar los trabajos analíticos y administrativos del sistema operativo.
3. **Responsabilidad en la Elección del Quantum:** En sistemas operativos para servidores de alta concurrencia, calibrar el quantum de manera deficiente deteriora el rendimiento global por exceso de conmutaciones de contexto o penaliza severamente a usuarios interactivos.

10. Limitaciones, Mejoras y Conclusiones

Limitaciones Identificadas:

- El simulador asume una arquitectura mononúcleo (1 sola CPU física).
- En sistemas operativos de producción, las ráfagas de CPU futuras en SRTF no se conocen de antemano; deben estimarse mediante promedios exponenciales ($\tau_{n+1} = \alpha t_n + (1-\alpha) \tau_n$).

Mejoras Futuras:

- Incorporar planificación multinúcleo (Symmetric Multiprocessing - SMP) con afinidad de CPU y balanceo de carga.
- Implementar colas multinivel con retroalimentación (MLFQ - Multi-Level Feedback Queue), que combina las virtudes de Round Robin y prioridades adaptativas sin requerir conocimiento previo de las ráfagas.

Conclusiones:

- No existe un algoritmo de planificación universalmente óptimo: FCFS maximiza el rendimiento bruto a costa de la equidad; Round Robin provee interactividad equitativa a costa de sobrecarga por cambio de contexto; y SRTF minimiza el tiempo de espera pero introduce riesgo de inanición.
- El modelado computacional y la recolección de métricas confirmaron con rigor empírico los principios teóricos fundamentales de los sistemas operativos.

---

11. Referencias

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
3. Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
4. Jain, R., Chiu, D. M., & Hawe, W. (1984). *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Systems*. DEC Research Report TR-301.